

学习资源

Agentic 扩展组件

 Copy page

理解在 Agentic Coding 中如何扩展能力：包括上下文文件、Skills、Subagent、外部工具连接以及自动化机制。

过去一年，AI 编程工具发生了一次非常明显的变化。早期的 AI 编程助手，比如 GitHub Copilot，更像是一个“智能自动补全”工具，开发者写代码时，AI 提供下一行建议。

现在，一个 Coding Agent 可以：



- 读取整个代码仓库
- 搜索和修改文件
- 运行测试和构建命令
- 调用外部 API
- 自动完成复杂开发任务

开发者不再只是问问题，而是可以直接给 AI 一个目标，例如：“给这个项目添加 OAuth 登录，并确保测试通过。”然后 AI 会规划步骤、修改代码、运行测试并不断迭代。这种模式被称为 **Agentic Coding**。

为了支持这种能力，各种 Coding Agent（例如 Claude Code、Cursor、Codex、Copilot Agent 等）逐渐形成了一个 **扩展组件层（Extension Layer）**。这些组件让 AI 不再只是一个模型，而更像一个可扩展的软件系统。

Coding Agent 的基本架构

在架构上，大多数 Coding Agent 都遵循类似的模式：

Model



Agent Loop



Tools

Extension Layer

└ Project Context files

└ Skills / workflows

└ Subagents

└ External tools

└ Hooks

└ Plugins

可以简单理解为：**模型负责思考，工具负责执行，而扩展组件层决定 Agent 的能力边界。**

研究者对近 3000 个使用 AI coding agent 的 GitHub 仓库进行分析发现，这些系统通常通过 **配置文件和扩展组件**来控制 Agent 的行为，例如 CLAUDE.md、Skills 和 Subagents 等。

Coding Agent 的能力，很大程度上来自于它的扩展组件。

Coding Agent 的能力不仅取决于模型本身，很大程度上也来自于 **扩展组件层** —— 即如何为 Agent 提供上下文信息、可复用能力模块以及外部工具连接。

在不同工具中，这些组件的名称和实现方式可能有所不同，但整体结构高度相似。为了更清晰地说明这一层级，下文将以 **Claude Code 的 Agentic 生态为参考**，介绍 Coding Agent 常见的几类扩展组件。

Claude Code 的扩展组件

Claude Code 的扩展组件可以嵌入到 **Agentic Loop 的不同阶段**，从而增强系统能力。主要扩展组件包括：

为 Coding Agent 提供每次会话都会加载的持久化上下文

▼ Skills

提供可复用的知识、指令和工作流程

▼ MCP (Model Context Protocol)

用于连接外部服务与工具系统

▼ Subagents

在独立上下文中运行子 Agent，并返回执行结果摘要

▼ Agent Teams

协调多个独立会话进行并行协作

▼ Hooks

在 Agent 循环之外执行确定性的自动化脚本

▼ Plugins

与 Marketplace 用于打包并分发上述扩展能力

其中，**Skills 是最灵活的一类扩展机制**。一个 Skill 本质上是一个 Markdown 文件，其中可以包含：领域知识、操作流程和指令说明。你还可以通过类似 `/deploy` 的命令直接调用某个 Skill；在某些情况下，Coding Agent 也会在任务相关时自动加载对应 Skill。

💡 Skills 既可以在当前会话中执行，也可以通过 **Subagent** 在隔离上下文中运行。

Claude Code 提供的扩展能力覆盖多种使用场景：从每次会话都会加载的上下文信息，到按需调用的能力模块，再到在特定事件触发的自动化流程。不同扩展组件的定位及适用场景如下表所示。

组件类型	作用	适用场景
CLAUDE.md	定义项目规则与背景信息，Coding Agent 每次开始工作时都会读取	项目规范、全局规则
Skills	把常用任务或知识整理成可复用的“能力模块”	参考文档、标准任务
Subagent	独立运行的子 Agent，用来处理复杂或耗时任务	上下文隔离、并行任务
Agent Teams	多个 Agent 协同工作，分工完成复杂任务	并行研究或复杂开发
MCP	让 Coding Agent 可以访问外部系统或工具	访问外部数据或执行外部操作
Hook	在特定事件发生时自动执行脚本	确定性自动化流程

Plugins

Plugin（插件） 是 Claude Code 的能力打包机制。一个插件可以同时包含 Skills、Hooks、Subagents 和 MCP Servers。

Plugins 中的 Skills 会使用 **命名空间**（例如 `/my-plugin:review` ），从而避免不同插件之间的命令冲突。

当你希望在多个仓库复用同一套配置并将能力分发给其他开发者时，可以通过 **Plugin 与 Marketplace** 来实现。

如何区分这些组件

部分扩展组件在功能上看起来相似，但它们解决的问题不同。下面对几种常见组合进行说明。

Skills vs Subagent	CLAUDE.md vs Skills	CLAUDE.md vs Rules vs Skills	MCP vs Skill
------------------------------------	-------------------------------------	--	------------------------------

定位不同：

维度	Skills	Subagent
本质	可复用知识或流程	独立执行单元
核心价值	在不同任务间复用内容	上下文隔离
适合场景	参考资料、标准流程	大规模代码分析或并行任务

Skills 可以分为两类：

1. **Reference Skills** 提供参考知识，例如 API 规范或开发指南。
2. **Action Skills** 触发具体任务，例如 `/deploy` 。

Subagent 更适合处理需要读取大量文件或执行复杂分析的任务。Subagent 在独立上下文中完成工作，并只向主会话返回摘要结果，从而避免占用主上下文。

 两者也可以组合使用：

- Subagent 可以预加载指定 Skills
- Skills 也可以在隔离上下文中执行

扩展组件的层级关系

扩展组件可以在多个层级定义，例如：

- 用户级
- 项目级
- 插件级
- 管理策略级

当同一种功能在多个层级存在时，系统会根据组件类型采用不同的合并或覆盖规则。

采用 **叠加 (additive)** 模式：所有层级的 CLAUDE.md 文件都会被加载，并合并到对话的上下文中。

当规则发生冲突时，通常是 **更具体的规则优先生效**。

组合使用扩展组件

在实际项目中，通常会组合使用多个扩展组件。例如：

组合方式	工作方式	示例
Skills + MCP	MCP 提供连接，Skills 定义使用方式	Skills 描述数据库 schema
Skills + Subagent	Skills 启动多个 Subagent	<code>/audit</code> 同时运行安全与性能检查
CLAUDE.md + Skills	全局规则 + 按需知识	CLAUDE.md 定义 API 规范
Hook + MCP	Hook 调用外部系统	修改关键文件后发送 Slack 通知

上下文成本

每一种扩展能力都会占用 **模型的上下文窗口**。

如果加载过多内容，可能导致：

- 模型忽略关键信息
- Skills 触发不准确
- Coding Agent 无法正确遵循项目规则

因此在设计扩展结构时，需要在能力与上下文成本之间取得平衡。

不同扩展组件的上下文成本

 BigModel	加载时机	加载内容	上下文成本	🔍	⋮
CLAUDE.md	会话开始	全部内容	每次请求		
Skills	会话开始 + 调用时	描述 + 完整内容	较低		
MCP servers	会话开始	工具定义	每次请求		
Subagents	创建时	独立上下文	与主会话隔离		
Hooks	触发时	默认无	0		

默认情况下，Skill 的描述会在会话开始时加载，以便 Coding Agent 判断是否需要使用该 Skill。

如果希望 Skill 仅在手动调用时加载，可以在 `frontmatter` 中设置：`disable-model-invocation: true`，这样 Coding Agent 在会话中不会看到该 Skill，只有在显式调用时才会加载，从而将上下文成本降为零。

学习资源



Coding Agent 工作原理



记忆机制



常用 workflow



最佳实践

>
